FOR COMPLETE BEGINNERS

Negative money?

Complete Modern Java, episode 9: number limits in Java and TypeScript

> Every number type has a limit. Know it, pick a bigger type, or make overflow stop the program.

Two billion cents plus two billion cents printed -294967296: negative money, no error.

An int has 32 bits and a fixed range. Past the top it wraps round: overflow.

In this episode

- Why an int overflows
- long, and the L suffix
- Math.addExact
- Dividing by zero
- TypeScript safe integers

Coming next

- Decisions: if and else (episode 10)
- and, or, not, else if (episode 11)
- Loops (episode 12)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each line printed

2_000_000_000 + itself (int)	`-294967296`
Integer.MAX_VALUE, then + 1	`2147483647, then -2147483648`
2_000_000_000L + itself (long)	`4000000000`
Math.addExact(MAX_VALUE, 1)	`ArithmeticException: integer overflow`
7.0 / 0 and 0.0 / 0	`Infinity and NaN`
10 / people (people = 0)	`ArithmeticException: / by zero`

Every row was run, not predicted.

02 Overflow, and what to do about it

An int stores 32 bits: -2,147,483,648 to 2,147,483,647. Past the top it wraps to the most negative value.

Use a long (L suffix) for bigger numbers, or Math.addExact to stop instead of returning a wrong number.

Limits.java

```
int cents = 2_000_000_000;
IO.println(cents + cents);            // -294967296
IO.println(Integer.MAX_VALUE + 1);    // -2147483648
long bigCents = 2_000_000_000L;
IO.println(bigCents + bigCents);      // 4000000000
```

> Printed exactly as commented.
>
> Run on a Mac, 2026-10-24: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

03 TypeScript

Numbers are doubles: no wrap at two billion, but past the largest safe integer whole numbers are not exact.

limits.ts

```
console.log(2_000_000_000 + 2_000_000_000);   // 4000000000
console.log(9007199254740992 + 1);            // 9007199254740992
console.log(10 / 0);                          // Infinity
console.log(0 / 0);                           // NaN
```

> npx tsc --noEmit was quiet; node printed the lines above.
>
> Run on a Mac, 2026-10-24: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

Before a number gets big

1. Know the range of the type you picked.
2. Bigger than about 2.1 billion? Use a long, with L.
3. A wrong answer worse than stopping? Math.addExact.
4. Dividing? Think about zero first.
5. Challenge: predict MAX_VALUE + 1, 2 billion L + itself, and 7.0 / 0.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-24 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.